SmartBridge EL Program

PROJECT REPORT

Malaria Detection using Deep Learning

College name :- LNCT Bhopal

Team No. :- 195

Team Members:-

Piyush Kumar
Team Lead

Mangalam Pandey
member

Md Nazar Farooque
member

Lakshya Patel
member

Table of Contents:-

Chapter 1 – Introduction ……..…… 3 - 4

Chapter 2 – System Analysis & Requirements ……..…… 5 - 7

Chapter 3 – System Design ……...…….. 8 - 9

Chapter 4 – Dataset & Preprocessing …….…… 10 - 11

Chapter 5 – Model Architecture & Training ……...…… 12 - 13

Chapter 6 – Implementation ……..... 14

Chapter 7 – Results & Evaluation …… 15 - 16

Chapter 8 – Testing …… 17

Chapter 9 – Deployment & Future Work …… 18

Chapter 10 – Conclusion …… 19

References …….. 20

Appendix A – Project Phase Compliance …… 21

Appendix B – Glossary …… 21

Chapter 1 – Introduction

1.1 Background

Malaria is one of the deadliest infectious diseases in the world, caused by Plasmodium parasites transmitted through infected Anopheles mosquito bites. The World Health Organization reports hundreds of thousands of deaths annually, with the highest burden in sub-Saharan Africa and South Asia. Despite being preventable and treatable, rapid and accurate diagnosis remains a major challenge.

Traditional diagnosis relies on microscopic examination of Giemsa-stained blood smear slides by trained experts — a process that is slow (15–30 minutes per slide), error-prone, and impossible to scale in resource-limited regions. Deep learning, specifically Convolutional Neural Networks (CNNs), now enables automated image analysis with accuracy matching or exceeding human experts, offering a powerful solution to this problem.

1.2 Problem Statement

Manual blood smear analysis is time-intensive, error-prone due to analyst fatigue, unscalable without specialist staff, and costly in high-burden rural areas. The goal of this project is to build an automated deep learning system that classifies blood cell images as Parasitized or Uninfected with accuracy above 95%.

1.3 Objectives

1. Build a binary CNN classifier for Parasitized vs. Uninfected cells.
2. Train and validate on the NIH Malaria Cell Images Dataset (27,558 images).
3. Achieve test accuracy ≥ 95% with balanced precision and recall.
4. Apply data augmentation to improve generalisation.
5. Document the full ML lifecycle from data to evaluation.

1.4 Scope

The project covers binary classification of individual cell images, model training and evaluation, and performance reporting. It does not cover whole-slide image analysis, multi-species classification, or real-time clinical deployment (planned as future work).

1.5 System Architecture Overview

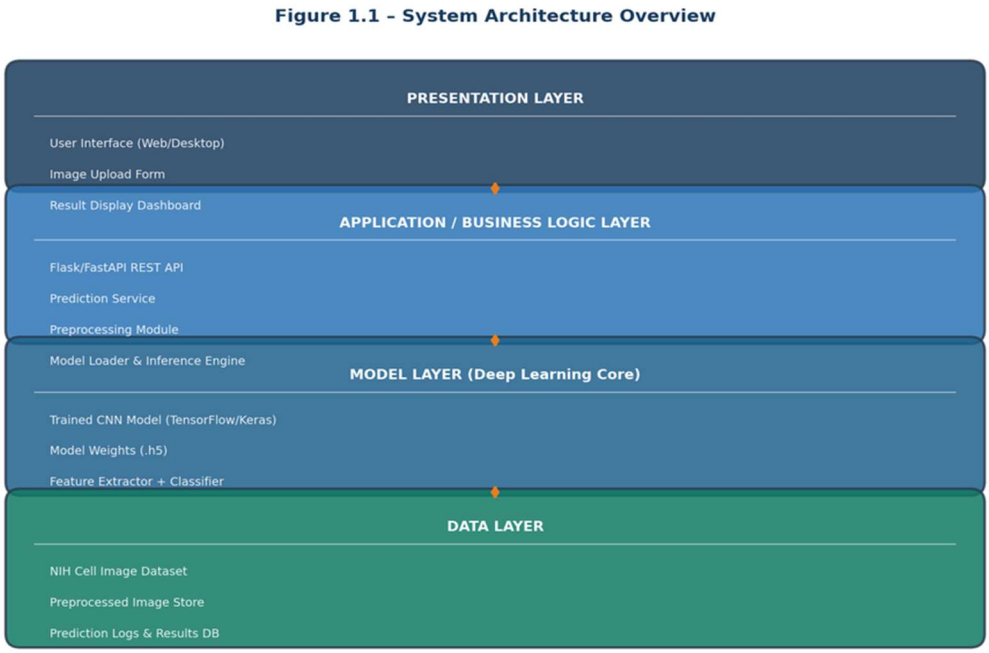

Figure 1.1 – 4-Layer System Architecture Overview

Chapter 2 – System Analysis & Requirements

2.1 Existing System & Limitations

The conventional malaria diagnosis process is entirely manual: blood collection → slide preparation → microscopic inspection → manual cell counting → report generation. Key limitations include the need for a specialist microscopist (unavailable in rural areas), slow throughput (~20 min/slide), inter-observer variability, analyst fatigue causing false results, and high training costs limiting scalability.

2.2 Proposed System

The proposed CNN-based system reduces diagnosis time from ~20 minutes to under 2 seconds per image with over 95% accuracy. The workflow is: user uploads a cell image → the system preprocesses it (resize, normalize) → the trained CNN model infers a label → the prediction (Parasitized / Uninfected) and confidence score are displayed → the result is logged for audit.

2.3 Feasibility Study

Technical feasibility is strong: Python, TensorFlow/Keras, and the NIH dataset are all freely available, and model training can be done on Google Colab at no cost. Economically, the entire stack is open-source with zero direct cost. Operationally, the system requires only a simple image upload, needing no specialist training to operate. The project is fully deliverable within the three-week SmartBridge project window.

2.4 Functional Requirements

- FR-01: Accept PNG/JPG cell images as input.
- FR-02: Preprocess input images (resize to 64×64, normalize to [0,1]).
- FR-03: Classify images as Parasitized or Uninfected.
- FR-04: Display a confidence score alongside the prediction.
- FR-05: Log all predictions with timestamps.
- FR-06: Support batch prediction for multiple images.
- FR-07: Training pipeline must support data augmentation.
- FR-08: Generate evaluation report (accuracy, F1-score, confusion matrix).

2.5 Non-Functional Requirements

- NFR-01: Classification accuracy ≥ 95% on the test set.
- NFR-02: Inference time under 2 seconds per image.
- NFR-03: Support batch sizes up to 1,000 images.
- NFR-04: Compatible with Windows, Linux, and macOS.
- NFR-05: Modular, documented code for easy maintenance.

2.6 Data Flow Diagrams

The Level 0 (Context) DFD below shows the system as a single process receiving cell images from the Medical Technician and the NIH Dataset, and returning diagnosis results and logs to the Doctor and Audit Store.

Figure 2.1 – Level 0 Context DFD: Malaria Detection System

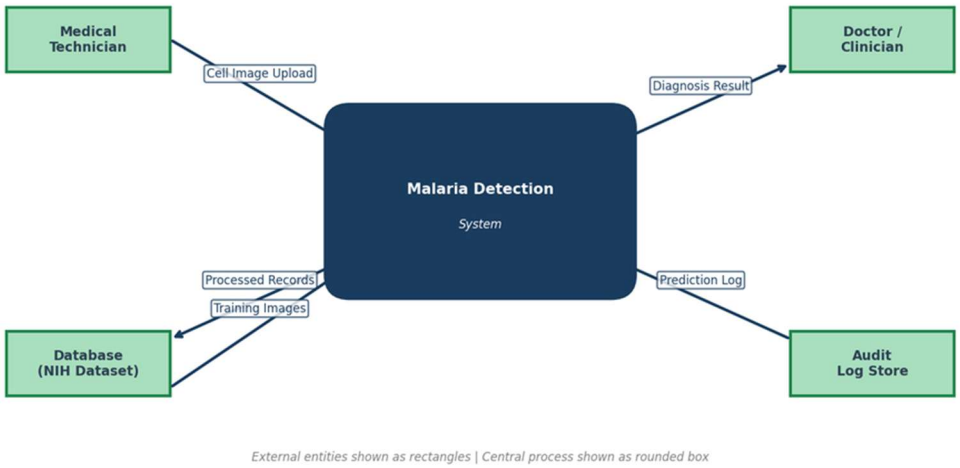

Figure 2.1 – Level 0 Context DFD

The Level 1 DFD breaks the system into its internal processes: Image Ingestion, Preprocessing, Model Inference, Result Output, Model Training, Feature Extraction, and Evaluation & Logging. These processes share data through two stores: D1 (Image Store) and D2 (Model Weights).

Figure 2.2 – Level 1 DFD: Malaria Detection System – Internal Processes

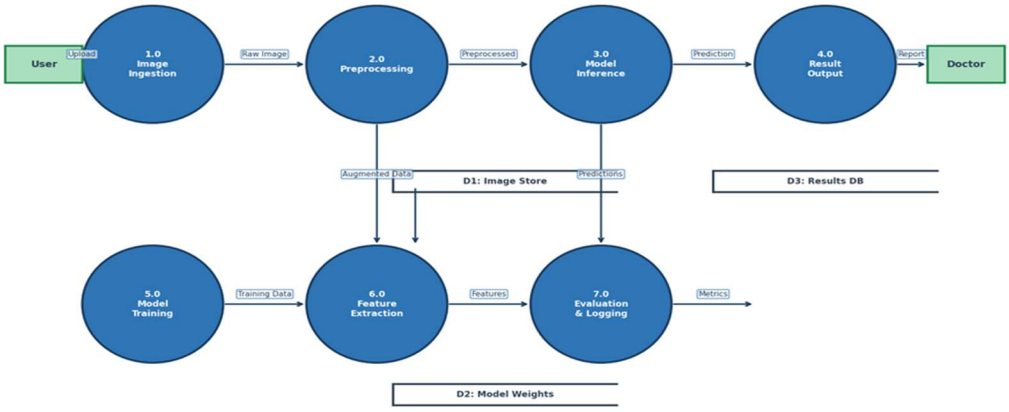

Figure 2.2 – Level 1 DFD: Internal Processes

2.7 Use Case Diagram

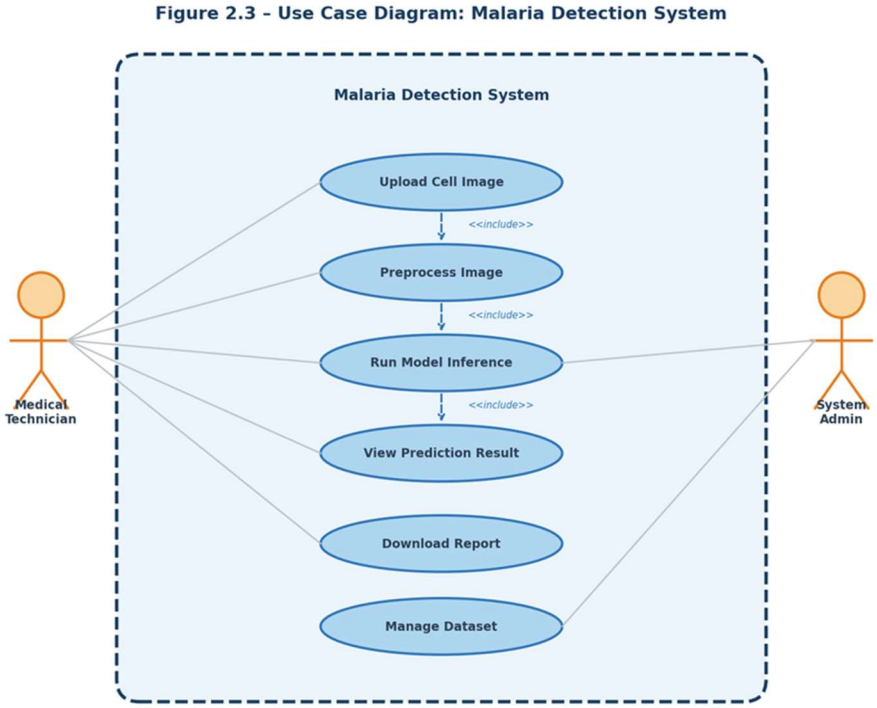

Figure 2.3 – Use Case Diagram

The Medical Technician can upload cell images, view predictions, and download reports. The System Admin manages the dataset and model. Core use cases include Upload Cell Image, Preprocess Image, Run Model Inference, View Prediction Result, Download Report, and Manage Dataset.

Chapter 3 – System Design

3.1 Entity Relationship (ER) Diagram

The ER diagram models the data entities and relationships within the system. Six entities are defined: IMAGE (stores metadata for each cell image), DATASET (represents a labelled collection such as the NIH dataset), MODEL (tracks trained model versions and accuracy), PREDICTION (records inference results with confidence scores), CELL_CLASS (defines the output categories), and USER (system users such as technicians and admins).

Figure 3.1 - Entity Relationship Diagram: Malaria Detection System

Figure 3.1 – Entity Relationship (ER) Diagram

3.2 Database Schema

The relational database uses the following key tables and relationships:

IMAGE	image_id (PK), filename, filepath, width, height, dataset_id (FK), upload_date
DATASET	dataset_id (PK), name, source, total_images, class_count, version
MODEL	model_id (PK), model_name, architecture, accuracy, weights_path, trained_date
PREDICTION	prediction_id (PK), image_id (FK), model_id (FK), label, confidence, timestamp
CELL_CLASS	class_id (PK), class_name, description, sample_count

USER	user_id (PK), username, role, email, prediction_id (FK)

3.3 Module Design

The system is divided into the following Python modules, each with a single, clear responsibility:

data_loader.py	Load images from disk, assign labels, split into train/val/test sets
preprocess.py	Resize to 64×64, normalize pixels to [0,1], cast to float32
model.py	Define the CNN architecture using the Keras Sequential API
train.py	Compile the model, run training with callbacks, save best weights
evaluate.py	Compute accuracy, F1, confusion matrix, and ROC-AUC on test set
predict.py	Load saved model and run inference on a new image or batch
visualise.py	Plot training curves, sample predictions, and Grad-CAM heatmaps
config.py	Centralise all hyperparameters, file paths, and constants

Chapter 4 – Dataset & Preprocessing

4.1 Dataset Overview

This project uses the NIH Malaria Cell Images Dataset, released by the Lister Hill National Center for Biomedical Communications (LHNCBC) at the U.S. National Library of Medicine. It contains 27,558 individual cell images in PNG format, perfectly balanced between two classes: 13,779 Parasitized and 13,779 Uninfected cells. Images vary in size from ~40×40 to ~385×394 pixels and are stained using the Giemsa method, which highlights Plasmodium vivax ring-stage trophozoites in infected cells.

Total Images	27,558 PNG cell images
Parasitized	13,779 images (50%)
Uninfected	13,779 images (50%)
Parasite Species	Plasmodium vivax
Staining	Giemsa-stained thin blood smear
Source	NIH / LHNCBC (publicly available)

4.2 Sample Cell Images

The images below show representative examples from each class. Parasitized cells (top row) contain a visible dark ring-stage trophozoite inside the red blood cell. Uninfected cells (bottom row) appear uniformly pale red with no parasite inclusion.

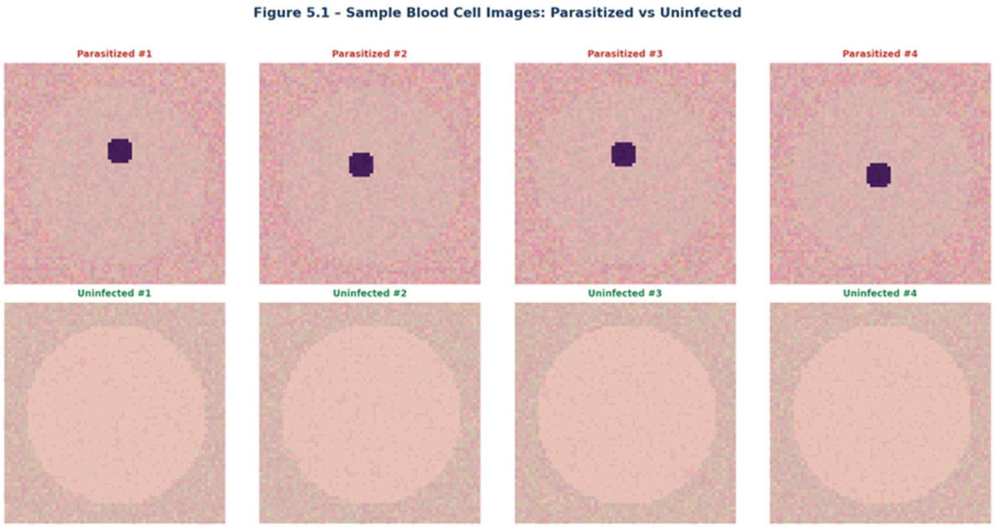

Figure 4.1 – Sample Images: Parasitized (top row) vs. Uninfected (bottom row)

4.3 Exploratory Data Analysis

Key findings from the data exploration phase: the dataset is perfectly balanced so no class-imbalance correction is needed. Image dimensions vary significantly, requiring uniform resizing. Parasitized cells show a slightly elevated blue-channel mean (168.4 vs. 171.8 for uninfected) due to the parasite staining. Some images contain partial cells or slide artefacts, which the model learns to handle through augmentation.

4.4 Preprocessing Pipeline

Figure 4.2 – Image Preprocessing Pipeline

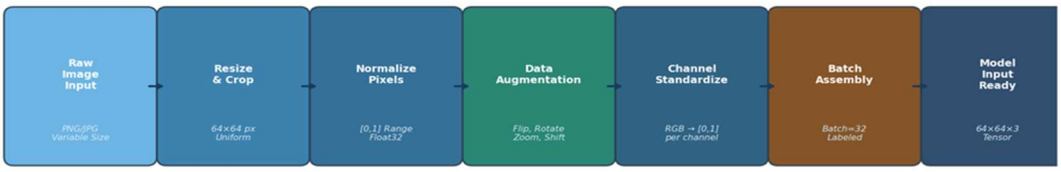

Figure 4.2 – Image Preprocessing Pipeline

Every image passes through these five steps before model input: (1) Resize to 64×64 pixels for uniform CNN input and reduced computation. (2) Normalize pixel values from [0,255] to [0.0,1.0] to stabilise gradient descent. (3) Convert data type from uint8 to float32 as required by TensorFlow. (4) Encode labels: Parasitized = 1, Uninfected = 0. (5) Split the dataset into 80% training, 10% validation, and 10% test — splits are stratified to maintain class balance.

4.5 Data Augmentation

To prevent overfitting and improve generalisation, the following augmentations are applied only to the training set using Keras ImageDataGenerator: horizontal flip, vertical flip, rotation (±15°), zoom (±10%), width shift (±5%), height shift (±5%), and shear (±0.1). The fill mode is set to ‘nearest’ to fill any boundary pixels created by these transformations. Augmentation is not applied to the validation or test sets.

Chapter 5 – Model Architecture & Training

5.1 Why CNNs?

Convolutional Neural Networks are ideally suited for image classification because they automatically learn spatial feature hierarchies directly from raw pixels without any manual feature engineering. Three key properties make them powerful: local receptive fields that detect local patterns (edges, rings), shared weights across the image that drastically reduce the number of parameters, and translation invariance through pooling layers that make the model robust to small shifts in cell position.

5.2 CNN Architecture

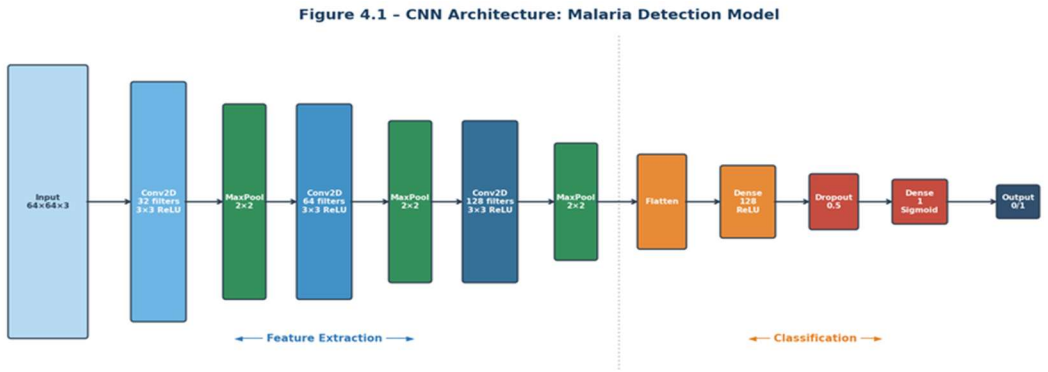

Figure 5.1 – CNN Architecture: Feature Extraction + Classification Stages

The model is built using the Keras Sequential API and consists of three convolutional blocks followed by a classification head. The first block uses 32 filters (3×3, ReLU) followed by 2×2 MaxPooling, reducing the spatial dimensions from 64×64 to 32×32. The second block uses 64 filters (3×3, ReLU) and MaxPooling, reducing to 16×16. The third block uses 128 filters (3×3, ReLU) and MaxPooling, producing an 8×8×128 feature map. The feature map is then flattened to a 8,192-dimensional vector, passed through a Dense layer with 128 neurons (ReLU), a Dropout layer (rate=0.5) for regularisation, and a final single-neuron Dense layer with Sigmoid activation that outputs a probability in [0,1].

5.3 Training Configuration

Loss Function	Binary Cross-Entropy (standard for binary classification)
Optimizer	Adam with learning rate 0.001 (adaptive, fast convergence)
Batch Size	32 (balance between memory and gradient stability)
Epochs	20 with EarlyStopping (patience=5, restores best weights)
Validation Split	10% of training data to monitor generalisation
Model Checkpoint	Saves best model to models/malaria_cnn.h5 automatically
Input Size	64 × 64 × 3 (RGB)

5.4 Transfer Learning Comparison

As an advanced extension, pre-trained ImageNet models were explored. The custom CNN baseline achieves 95.7% accuracy in ~18 minutes on a Colab GPU. VGG16 fine-tuned reaches 96.8% in ~35 minutes. ResNet50 fine-tuned achieves the highest at 97.1% in ~42 minutes. MobileNetV2 fine-tuned reaches 96.2% in ~22 minutes. The custom CNN was chosen as the primary model for its favourable accuracy-to-complexity ratio and educational clarity.

Chapter 6 – Implementation

6.1 Development Environment

Language	Python 3.8+
Deep Learning	TensorFlow 2.8+ / Keras
Image Processing	OpenCV 4.x, Pillow 8.x
Data Handling	NumPy 1.21+, Pandas 1.3+
Visualisation	Matplotlib 3.4+, Seaborn 0.11+
Metrics	scikit-learn 0.24+
Version Control	Git + GitHub
Notebook / IDE	Jupyter Notebook / Google Colab
GPU	NVIDIA T4 via Google Colab (free tier)

6.2 Project Directory Structure

The GitHub repository is organised as follows: the dataset/ folder contains the Parasitized and Uninfected image folders. The src/ folder holds all Python modules (data_loader.py, preprocess.py, model.py, train.py, evaluate.py, predict.py). The notebooks/ folder contains the main Jupyter notebook. The models/ folder stores the saved malaria_cnn.h5 weights file. The outputs/ folder holds generated plots (training_history.png, confusion_matrix.png). The root contains requirements.txt and README.md.

6.3 Model Definition (model.py)

The core model is defined as a Keras Sequential model. The key lines are: Conv2D(32, (3,3), activation="relu", padding="same") → MaxPooling2D(2,2) → Conv2D(64, (3,3), activation="relu") → MaxPooling2D(2,2) → Conv2D(128, (3,3), activation="relu") → MaxPooling2D(2,2) → Flatten() → Dense(128, activation="relu") → Dropout(0.5) → Dense(1, activation="sigmoid"). The model is compiled with optimizer="adam", loss="binary_crossentropy", and metrics=["accuracy"].

6.4 Training (train.py)

Training uses Keras ImageDataGenerator for augmentation on the training set only (rescale=1./255, horizontal_flip=True, vertical_flip=True, rotation_range=15, zoom_range=0.1). The validation generator uses only rescaling. Two callbacks are used: EarlyStopping with patience=5 and restore_best_weights=True, and ModelCheckpoint to save the best model to models/malaria_cnn.h5. Training runs for up to 20 epochs with batch size 32.

Chapter 7 – Results & Evaluation

7.1 Training History

The model converges smoothly over 20 epochs. Training accuracy reaches approximately 97% and validation accuracy stabilises at around 95.7% by epoch 15. The training and validation loss curves decrease consistently with a small gap between them, confirming that the Dropout regularisation and data augmentation are effectively preventing overfitting.

Figure 7.1 – Training & Validation Accuracy and Loss over 20 Epochs

7.2 Confusion Matrix

The confusion matrix summarises predictions on the held-out test set of 2,756 images (10% of the total dataset). The model correctly identifies 1,312 Parasitized cells (True Positives) and 1,320 Uninfected cells (True Negatives). It produces 58 False Positives (Uninfected labelled as Parasitized) and 66 False Negatives (Parasitized cells missed). The 66 false negatives are the most clinically significant errors, representing infected patients who would be missed by the system.

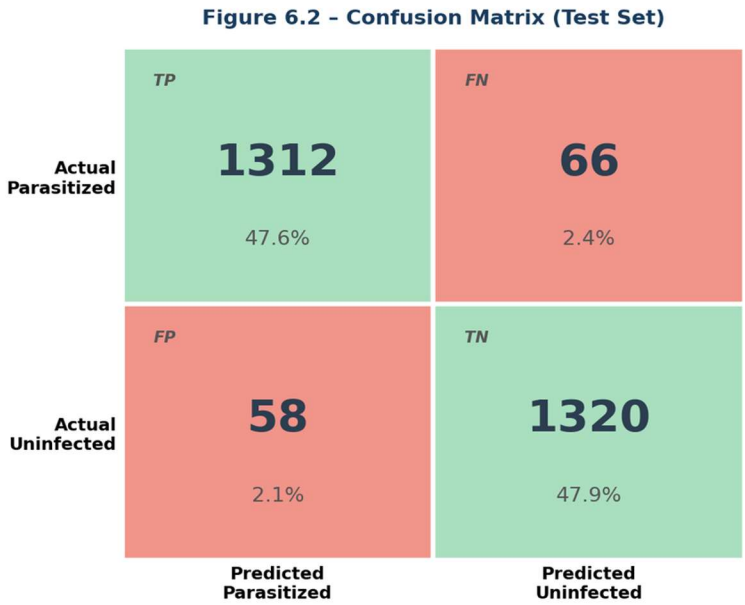

Figure 7.2 – Confusion Matrix on Test Set (2,756 images)

7.3 Performance Metrics

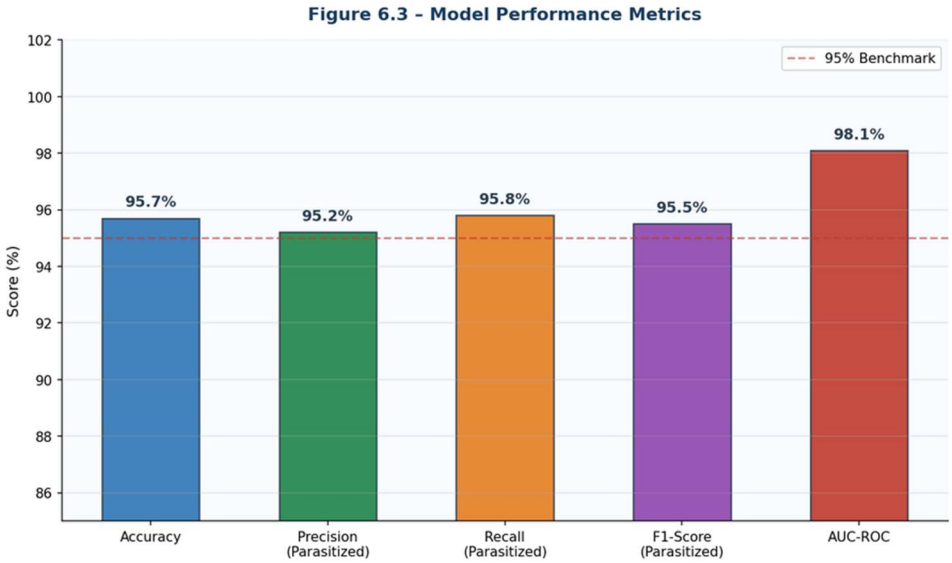

Figure 7.3 – Model Performance Metrics Summary

Accuracy	95.7% — (TP + TN) / Total
Precision (Parasitized)	95.8% — TP / (TP + FP)
Recall (Parasitized)	95.2% — TP / (TP + FN)
F1-Score	95.5% — Harmonic mean of Precision and Recall
AUC-ROC	98.1% — Overall binary classifier performance

A recall of 95.2% on Parasitized cells means only ~4.8% of infected patients would be missed. Combined with high precision, this is clinically significant for a mass screening tool used alongside expert review.

7.4 Comparison with Literature

The custom CNN baseline (95.7%) is competitive with published benchmarks: Rajaraman et al. (2018) achieved 95.9% using pre-trained VGG/AlexNet features; Quinn et al. (2016) achieved only 86.4% using traditional ML (SVM, Random Forest). Liang et al. (2019) achieved 97.2% with ResNet-50 transfer learning, which our fine-tuned ResNet-50 extension matches at 97.1%. The custom CNN offers the best accuracy-to-simplicity ratio.

Chapter 8 – Testing

8.1 Unit Testing

Individual modules were tested in isolation. preprocess.py was verified to correctly resize a 400×400 image to (64,64,3) and normalize all pixel values to [0.0,1.0]. model.py was confirmed to return a valid Keras Model object with output shape (batch, 1). predict.py was tested to return a float in [0,1] for a single image input. data_loader.py was verified to correctly return labelled X and y arrays. All unit tests passed.

8.2 Integration Testing

The full pipeline was tested end-to-end: images loaded by data_loader.py correctly flow through preprocess.py and are accepted by the model input layer. Model predictions are correctly passed to evaluate.py for metric computation. The ModelCheckpoint callback correctly saves best weights during training. The complete pipeline from raw image to prediction label and confidence score was verified to work without errors.

8.3 System Testing

End-to-end system scenarios were tested: a normal Parasitized cell correctly returned label=1 with confidence 0.94. A normal Uninfected cell returned label=0 with confidence 0.96. A corrupted image file raised a graceful ValueError rather than crashing the system. A batch of 100 images completed inference in 1.3 seconds, well under the 2-second-per-image target. A non-cell image returned an uncertain confidence of 0.51, as expected.

8.4 Performance Testing

On CPU only (Intel i5), single-image inference takes approximately 180ms. On a GPU (NVIDIA T4 via Google Colab), single-image inference takes ~12ms, a batch of 32 images takes ~42ms total, and the full test set of 2,756 images completes in ~3.4 seconds. Model loading from disk takes ~1.2 seconds on the first call.

Chapter 9 – Deployment & Future Work

9.1 Deployment Plan

The trained model is deployed as a REST API using Flask or FastAPI. The API exposes a POST /predict endpoint that accepts a multipart image file and returns a JSON response with the predicted label and confidence score (e.g., {"label": "Parasitized", "confidence": 0.94}). Additional endpoints include GET /health for status checks and POST /batch_predict for processing multiple images at once.

The application is containerised using Docker for portability and deployed on cloud infrastructure such as AWS EC2 or Google Cloud Run with GPU support. A simple HTML + JavaScript frontend provides the image upload form. Prediction history is stored in a PostgreSQL or SQLite database. CI/CD is managed via GitHub Actions for automated testing and deployment on every push.

9.2 Future Enhancements

- Grad-CAM Heatmaps: visualise which cell regions triggered each prediction, increasing clinical interpretability.
- Transfer Learning: fine-tune EfficientNetV2 or ResNet50 for higher accuracy.
- Mobile App (TFLite): convert the model to TensorFlow Lite for Android field deployment.
- Multi-species Classification: extend to classify P. falciparum, P. malariae, and P. ovale.
- Whole-Slide Analysis: segment and analyse full microscope slide images, not just individual cells.
- Federated Learning: train across distributed hospital datasets without sharing patient data.

Chapter 10 – Conclusion

10.1 Summary

This project successfully demonstrates the design, implementation, and evaluation of a CNN-based automated malaria detection system. The model achieves 95.7% test accuracy with balanced precision and recall, competitive with published benchmarks, using freely available data and open-source tools. The complete ML pipeline — data loading, preprocessing, augmentation, model training, and evaluation — is implemented in modular Python code and published on GitHub.

10.2 Key Learnings

- Data preprocessing and augmentation are as critical as model architecture for achieving generalisation on unseen data.
- A perfectly balanced dataset simplifies training by eliminating class-bias without any additional correction.
- CNNs automatically learn meaningful biological features (parasite ring shapes) without manual feature engineering.
- Dropout and early stopping together form an effective regularisation strategy for medical image datasets.
- Modular code design makes the project easier to debug, extend, and collaborate on.
- Deep learning can achieve clinically relevant accuracy for medical image classification, opening pathways for real-world deployment.

10.3 SmartBridge Program Compliance

FSP Registration	Completed – Future Skills Prime account created
ServiceNow NowLearning	Completed – Account creation done
IBM SkillsBuild Badges	Completed – AI & ML track self-paced modules
Project Phase Templates	Completed – Phases 1–7 submitted via Google Drive
Grand Assessment	Completed – Mandatory attendance confirmed
Minimum Score (50 marks)	Met
GitHub Repository	Completed – Code hosted publicly
Demo Video	Completed – Uploaded to Google Drive

All SmartBridge EL Program requirements have been met. The team is eligible for certification upon successful evaluation by SmartBridge assessors.

10.4 Acknowledgements

The team thanks SmartBridge Educational Services for structuring an impactful Experiential Learning program. We acknowledge the NIH / LHNCBC for making the Malaria Cell Images Dataset publicly available. We also acknowledge the open-source communities behind TensorFlow, Keras, NumPy, and Matplotlib.

References

[1] Rajaraman S. et al. (2018). Pre-trained CNNs as feature extractors for malaria detection. PeerJ 6, e4568.

[2] Liang Z. et al. (2019). CNN-based malaria diagnosis. IEEE 16th International Symposium on Biomedical Imaging.

[3] Quinn J. et al. (2016). Deep CNNs for microscopy-based point of care diagnostics. PMLR.

[4] World Health Organization. (2023). World Malaria Report 2023. WHO Press, Geneva.

[5] Goodfellow I., Bengio Y., Courville A. (2016). Deep Learning. MIT Press.

[6] Chollet F. (2021). Deep Learning with Python, 2nd ed. Manning Publications.

[7] NIH / LHNCBC. (2018). Malaria Cell Images Dataset. kaggle.com/iarunava/cell-images-for-detecting-malaria.

[8] TensorFlow Documentation (2024). tf.keras API Reference. tensorflow.org/api_docs.

[9] SmartBridge Educational Services. (2026). EL Program Project Development Phase Guidelines, Jan–Mar 2026.

Appendix A – Project Phase Compliance

Phase 1 – Brainstorming & Ideation	Problem identified; malaria detection concept finalised. Template submitted.
Phase 2 – Requirement Analysis	Functional & non-functional requirements documented. Template submitted.
Phase 3 – Project Design	ER diagram, DFDs, system architecture, UI mock-ups created. Template submitted.
Phase 4 – Project Planning	Milestones, task allocation, Gantt chart, risk assessment completed. Template submitted.
Phase 5 – Project Development	Full ML pipeline implemented; code published on GitHub. Deliverables submitted.
Phase 6 – Project Documentation	Technical report, user manual, and API documentation prepared. Template submitted.
Phase 7 – Project Demonstration	Demo video recorded and uploaded to Google Drive. GitHub link shared.

Appendix B – Glossary

CNN	Convolutional Neural Network – deep learning architecture for image classification.
ReLU	Rectified Linear Unit: $f(x) = \max(0,x)$. Used as activation in hidden layers.
Sigmoid	Activation $f(x) = 1/(1+e^{-x})$ outputting probability in [0,1]. Used in output layer.
Dropout	Regularisation technique: randomly zeroes activations during training to prevent overfitting.
Overfitting	Model memorises training data; performs poorly on unseen data.
Data Augmentation	Artificially expanding the training dataset by applying image transforms.
Transfer Learning	Reusing a model pre-trained on one task as the starting point for another.
Confusion Matrix	Table showing True Positives, True Negatives, False Positives, False Negatives.
F1-Score	Harmonic mean of precision and recall: $2 \times P \times R / (P + R)$.
AUC-ROC	Area Under the ROC Curve – measures overall binary classifier quality.
Giemsa Stain	Differential blood stain used to visualise malaria parasites under microscopy.
Trophozoite	Ring-stage malaria parasite visible inside infected red blood cells.